



WELCOME





C++ STL: Queue & Priority Queue

Agenda

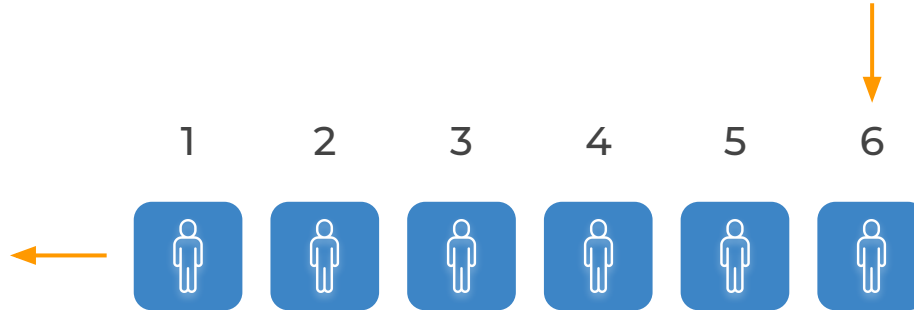
C++ Bridge Course

Queue

Priority Queue

Introduction

Queue follows the **FIFO** (**First-In, First-Out**) principle, meaning elements are added at the **back** and removed from the **front**.

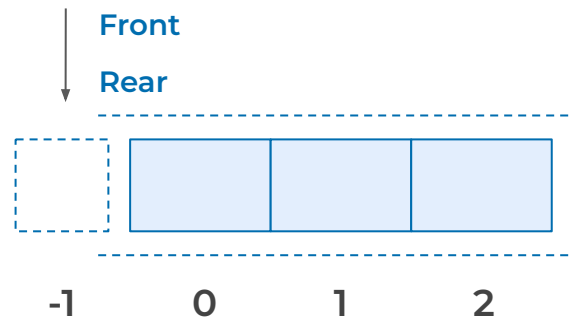


Declaration and Initialization

C++ 

```
queue<int> q;
```

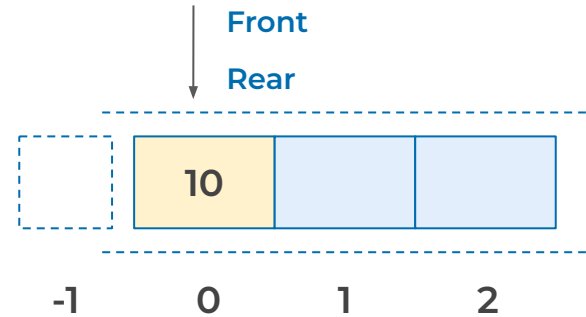
Creates an empty queue.



Adding Element

C++ 

```
queue<int> q;  
q.push(10);
```

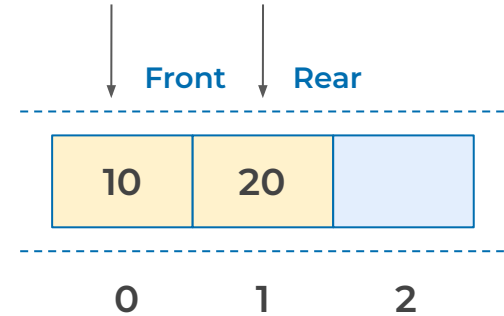


Adding Elements to the Queue.

Adding Element

C++ 

```
queue<int> q;  
q.push(10);  
q.push(20);
```

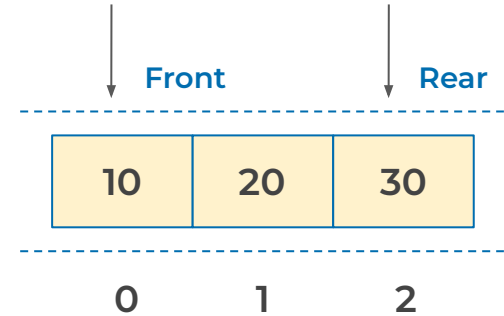


Adding Elements to the Queue.

Adding Element

C++ 

```
queue<int> q;  
q.push(10);  
q.push(20);  
q.push(30);
```



Adding Elements to the Queue.

Removing Element

C++ 

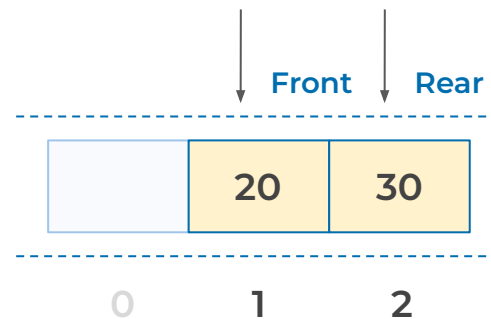
```
queue<int> q;
```

```
•
```

```
•
```

```
q.pop();
```

Removes the front element (10).

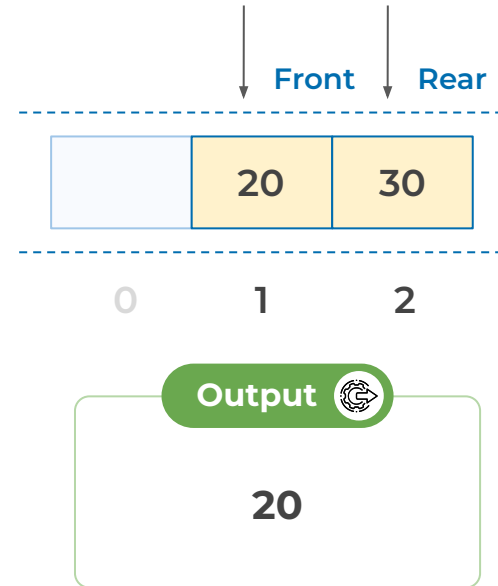


Accessing Front Element

C++ 

```
queue<int> q;  
.  
q.pop();  
cout << q.front();
```

Outputs 20 (Front element).

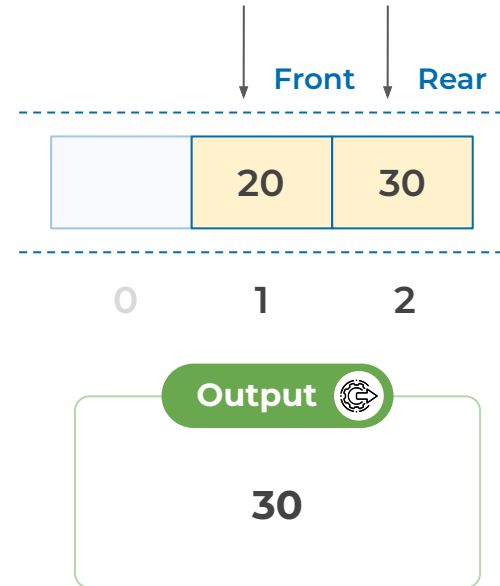


Accessing Back Element

C++ 

```
queue<int> q;  
.  
.  
q.pop();  
cout << q.front();  
cout << q.back();
```

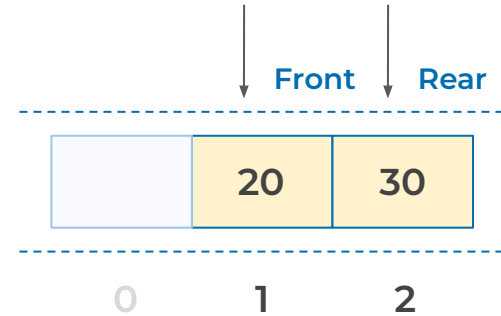
Outputs 30 (Back element).



Checking if the Queue is Empty

C++ 

```
queue<int> q;  
.  
.  
.  
if (q.empty()) {  
    cout << "Queue is empty";  
}  
else {  
    cout << "Queue is not empty";  
}
```

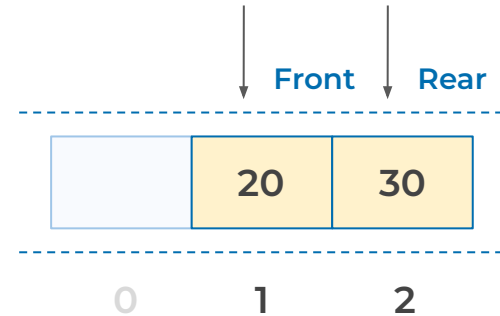


q.empty() - returns **1** if **queue** is **empty** otherwise returns **0**

Checking if the Queue is Empty

C++ 

```
queue<int> q;  
.  
.  
.  
if (q.empty()) {  
    cout << "Queue is empty";  
}  
else {  
    cout << "Queue is not empty";  
}
```



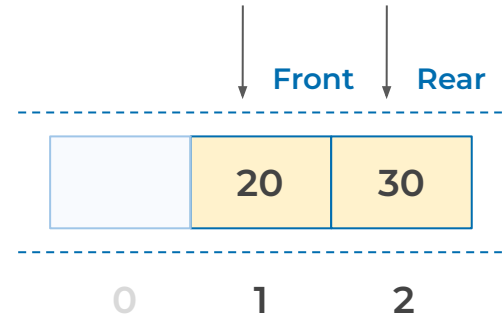
Output 

Queue is not empty

Getting the Size of the Queue

C++ 

```
queue<int> q;  
.  
.  
.  
cout << q.size();
```



q.size() returns the elements present in Queue

Getting the Size of the Queue

C++ 

```
queue<int> q;
```

```
·  
·  
·
```

```
cout << q.size();
```

Output 

2

Quiz Time!



Queue Manager

- ✓ You are tasked with implementing a program to **manage a queue of integers** using the C++ STL queue container. The program should support the following operations:

enqueue x: Add an integer to the back of the queue.

dequeue: Remove an integer from the front of the queue.

product: Calculate and return the product of the front and back elements of the queue.

size: Return the number of elements currently in the queue.

Example

Input:

enqueue 10

enqueue 20

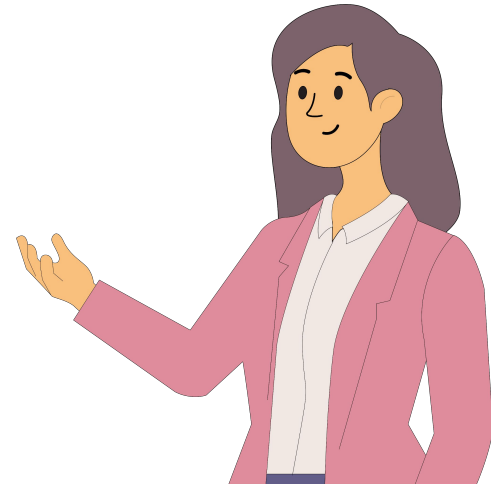
size

enqueue 30

product

dequeue

product



Example

Explanation:

enqueue 10

enqueue 20

size

enqueue 30

product

dequeue

product

Queue



enqueued "10" into Queue

Example

Explanation:

enqueue 10

enqueue 20

size

enqueue 30

product

dequeue

product

Queue



enqueued "20" into Queue

Example

Explanation:

enqueue 10

enqueue 20

size

enqueue 30

product

dequeue

product

Queue



Size of the queue is “2”

Example

Explanation:

enqueue 10

enqueue 20

size

enqueue 30

product

dequeue

product

Queue



enqueued "30" into Queue

Example

Explanation:

enqueue 10

enqueue 20

size

enqueue 30

product

dequeue

product

Queue



Product of front and back
elements: "300"

Example

Explanation:

enqueue 10

enqueue 20

size

enqueue 30

product

dequeue

product

Queue



dequeued "10" from Queue

Example

Explanation:

enqueue 10

enqueue 20

size

enqueue 30

product

dequeue

product

Queue



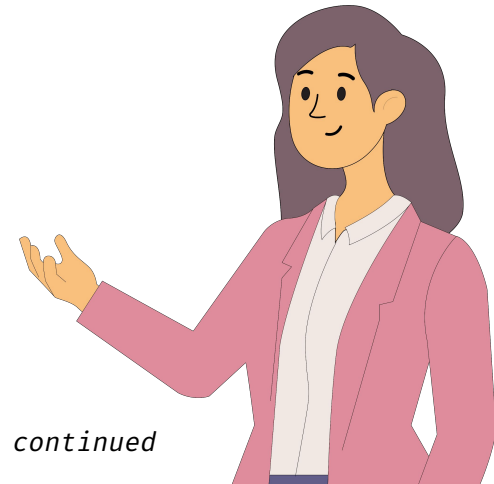
Product of front and back
elements: "600"

Example

C++ 

```
#include <bits/stdc++.h>
using namespace std;
class QueueManager {
public:
    queue<int> q;
    void enqueue(int value) {
        q.push(value);
        cout << value << " has been enqueued." << endl;
    }
    void dequeue() {
        if (!q.empty()) {
            q.pop();
            cout << q.front() << " has been dequeued." << endl;
        } else {
            cout << "Queue is empty. Cannot dequeue." << endl;
        }
    }
}
```

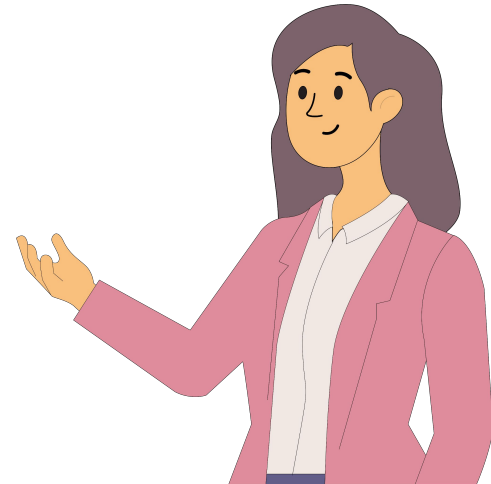
continued

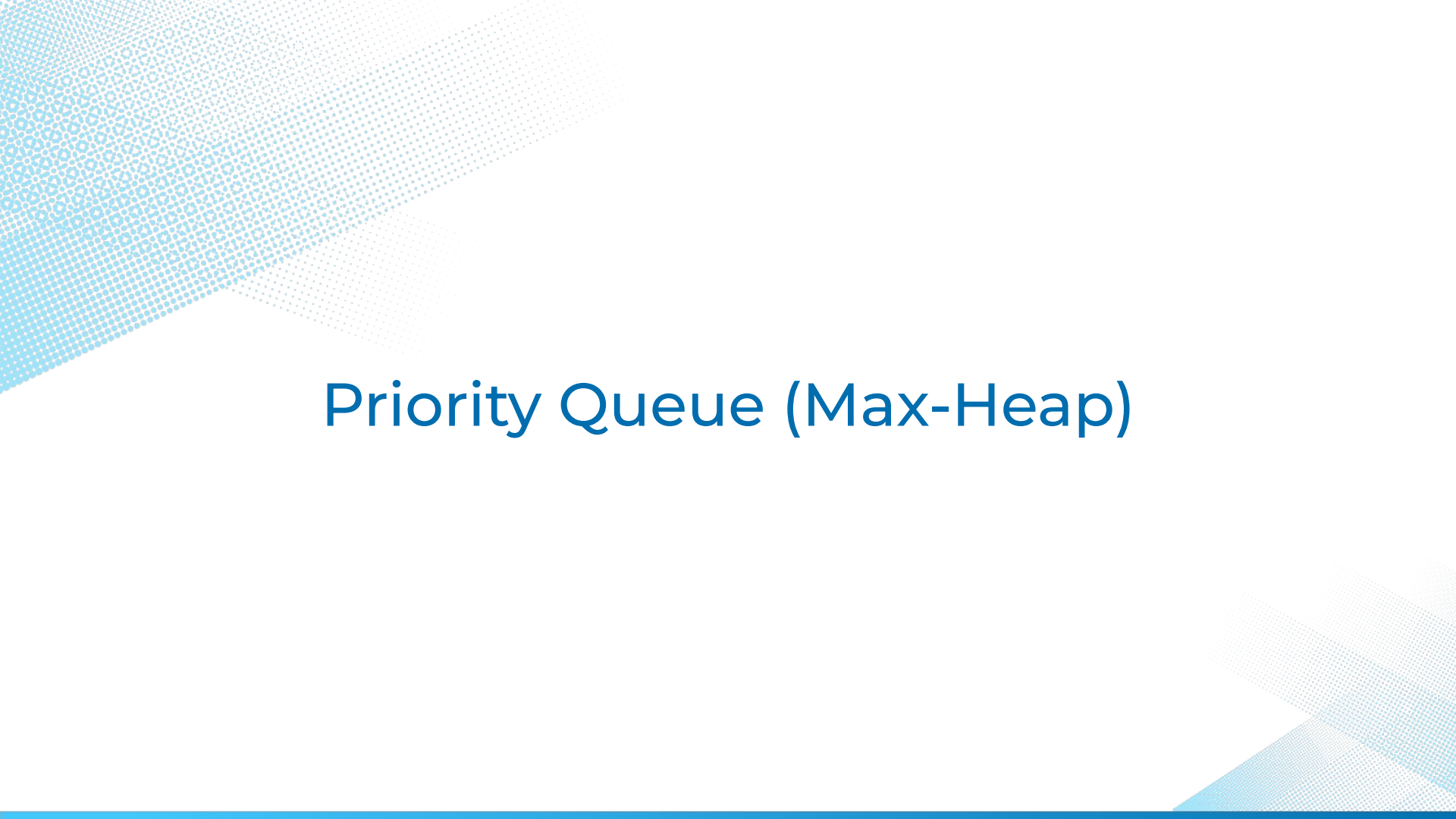


Example

C++ 

```
void productOfFrontAndBack() {  
    if (!q.empty()) {  
        cout << "Product of front and back elements: " <<  
q.front() * q.back() << endl;  
    } else {  
        cout << "Queue is empty. No product to compute." <<  
endl;  
    }  
}  
  
void size() {  
    cout << "Size of the queue: " << q.size() << endl;  
}  
};
```

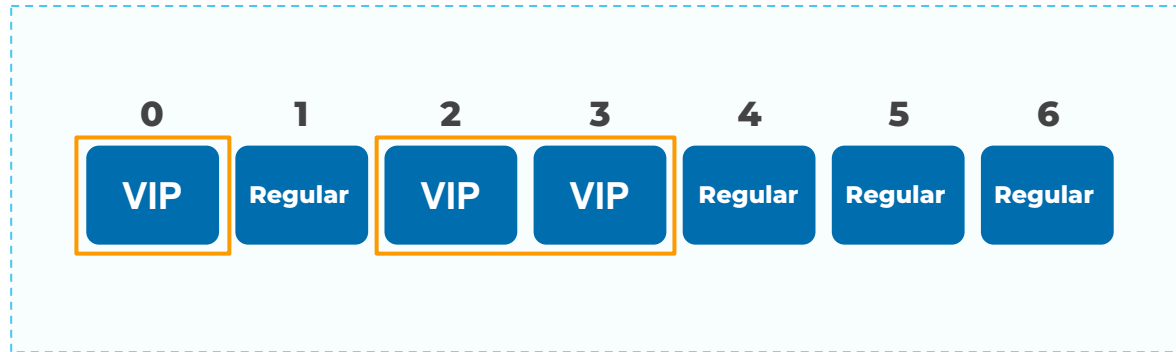




Priority Queue (Max-Heap)

Max-Heap

A **priority queue** is a type of queue where elements are **served based on priority** rather than on the order in which they arrive.



*So, **VIP** are prioritized first,
irrespective of the order of insertion.*

Max-Heap

It is a special type of tree-based data **structure** where the largest value is always at the top (root node of tree).

Example

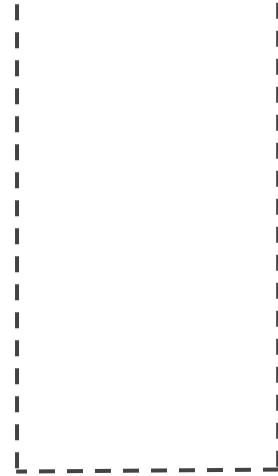
- **50** is the largest element.
- Removing the top (**50**) moves the next greater element (**30**) to the top.

Max-Heap

C++ 

```
priority_queue<int> pq;
```

Default max-heap



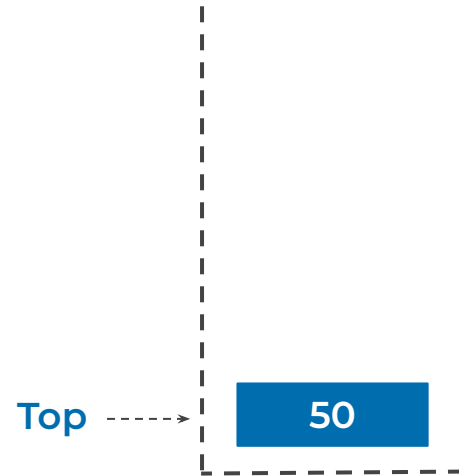
Priority Queue (Max-Heap)

Insert Elements

C++ 

```
priority_queue<int> pq;  
pq.push(50);
```

Element “50” is added to Priority queue.



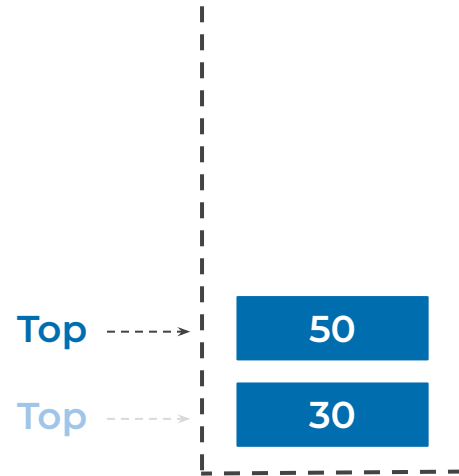
Priority Queue (Max-Heap)

Insert Elements

C++ 

```
priority_queue<int> pq;  
pq.push(50);  
pq.push(30);
```

Largest element is at the top.



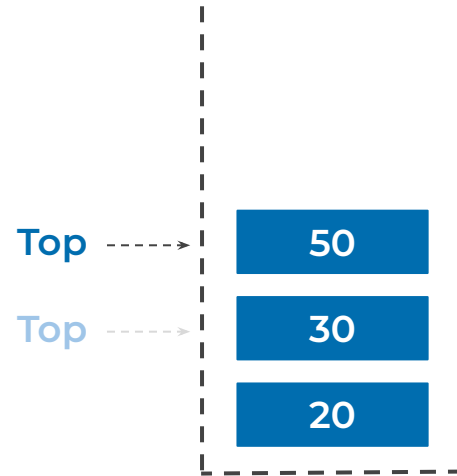
Priority Queue (Max-Heap)

Insert Elements

C++ 

```
priority_queue<int> pq;  
pq.push(50);  
pq.push(30);  
pq.push(20);
```

Largest element is at the top.



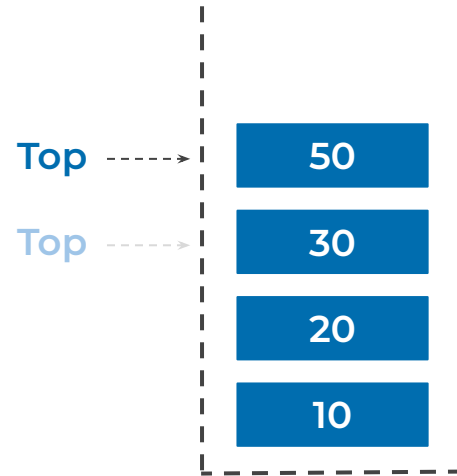
Priority Queue (Max-Heap)

Insert Elements

C++ 

```
priority_queue<int> pq;  
pq.push(50);  
pq.push(30);  
pq.push(20);  
pq.push(10);
```

Largest element is at the top.



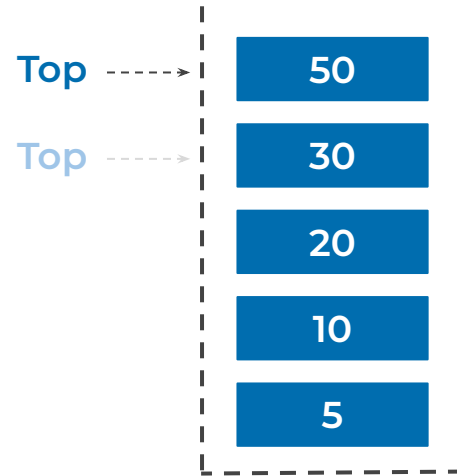
Priority Queue (Max-Heap)

Insert Elements

C++ 

```
priority_queue<int> pq;  
pq.push(50);  
pq.push(30);  
pq.push(20);  
pq.push(10);  
pq.push(5);
```

Largest element is at the top.



Priority Queue (Max-Heap)

Remove the Top Element

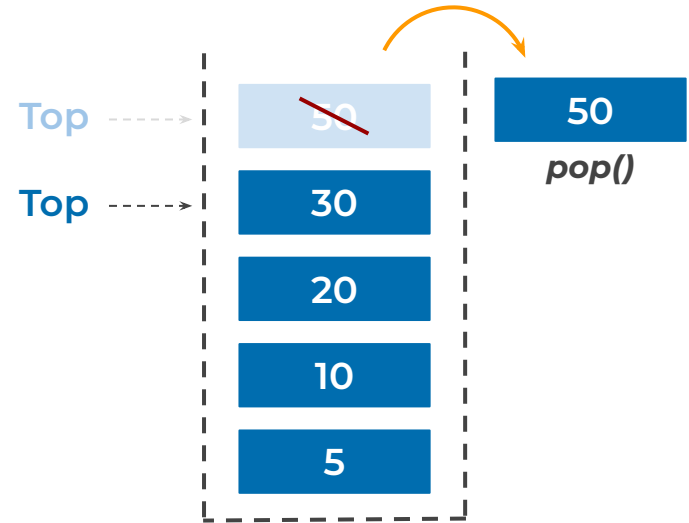
C++ 

```
priority_queue<int> pq;
```

```
•  
•  
•
```

```
pq.pop();
```

Removes the top element "50".



Priority Queue (Max-Heap)

Get the Top Element

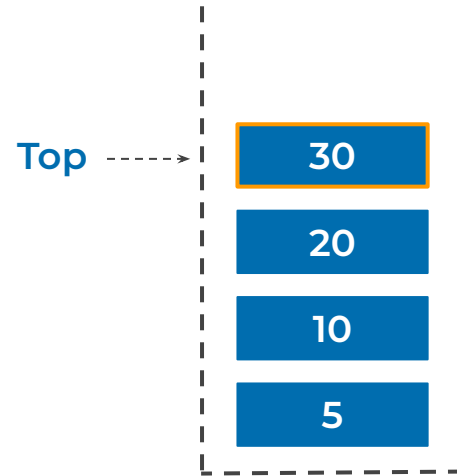
C++ 

```
priority_queue<int> pq;
```

•
•
•

```
cout << pq.top();
```

Get highest priority element.



Priority Queue (Max-Heap)

Get the Top Element

C++ 

```
priority_queue<int> pq;
```

•
•
•

```
cout << pq.top();
```

Get highest priority element.

Output 

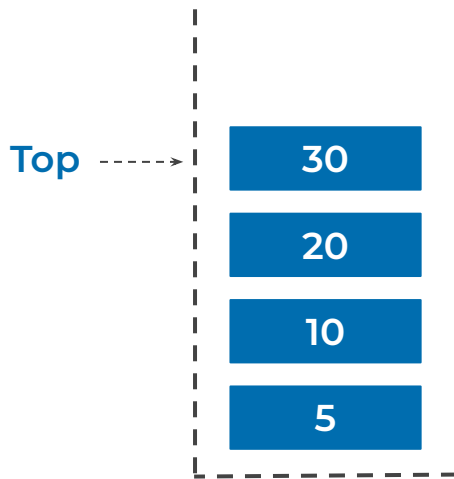
30

Check if the Priority Queue is Empty

C++ 

```
priority_queue<int> pq;  
:  
:  
if (pq.empty()) {  
    cout << "Priority queue is empty";  
}  
else {  
    cout << "Priority queue is not empty";  
}
```

***pq.empty()** returns 1 if it is empty, else returns 0*



Priority Queue (Max-Heap)

Check if the Priority Queue is Empty

C++ 

```
priority_queue<int> pq;  
:  
:  
:  
if (pq.empty()) {  
    cout << "Priority queue is empty";  
}  
else {  
    cout << "Priority queue is not empty";  
}
```

Output 

Priority queue is
not empty

`pq.empty()` - returns 1 if it is empty, else returns 0

Get the Size of the Priority Queue

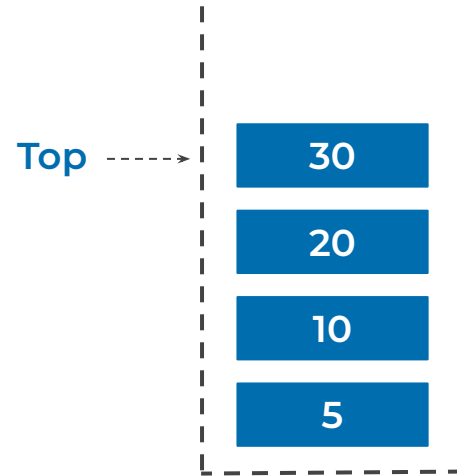
C++ 

```
priority_queue<int> pq;
```

•
•
•

```
cout << pq.size();
```

Outputs the number of elements in the priority queue.



Priority Queue (Max-Heap)

Get the Size of the Priority Queue

C++ 

```
priority_queue<int> pq;
```

```
•  
•  
•
```

```
cout << pq.size();
```

Output 

4

Outputs the number of elements in the priority queue.

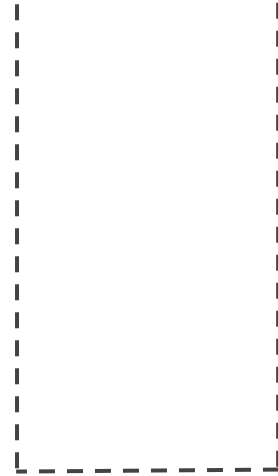


Priority Queue (Min-Heap)

Min-Heap

C++ 

```
priority_queue<int, vector<int>, greater<int>> pq;
```



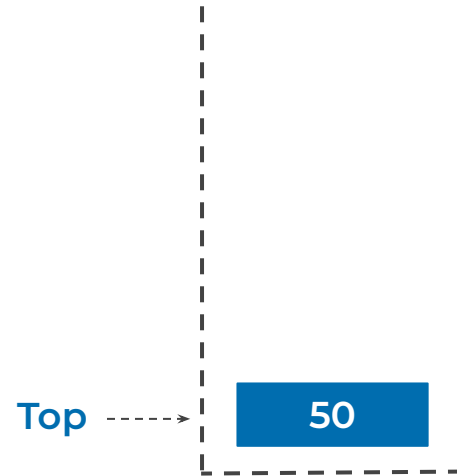
*A Min-Heap ensures that the **smallest element** is
always at the **top** and is **used first**.*

Insert Elements

C++ 

```
priority_queue<int, vector<int>, greater<int>> pq;  
pq.push(50);
```

Element “50” is added to the priority queue.



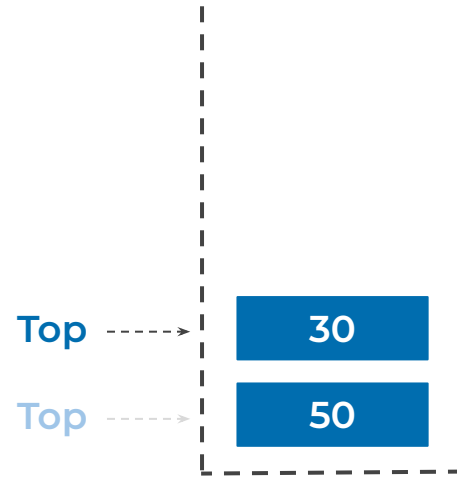
Priority Queue (Min-Heap)

Insert Elements

C++ 

```
priority_queue<int, vector<int>, greater<int>> pq;  
pq.push(50);  
pq.push(30);
```

Smallest element is at the top.



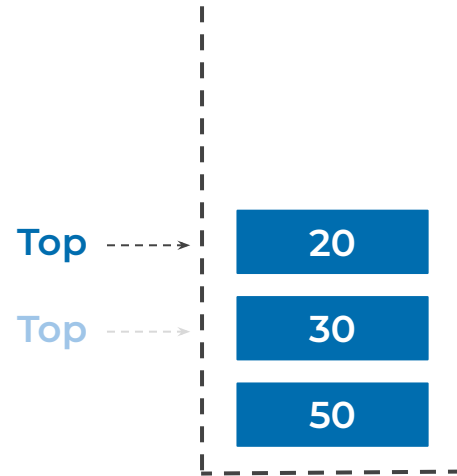
Priority Queue (Min-Heap)

Insert Elements

C++ 

```
priority_queue<int, vector<int>, greater<int>> pq;  
pq.push(50);  
pq.push(30);  
pq.push(20);
```

Smallest element is at the top.



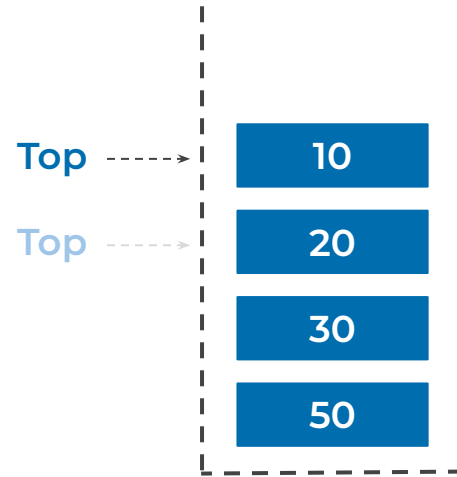
Priority Queue (Min-Heap)

Insert Elements

C++ 

```
priority_queue<int, vector<int>, greater<int>> pq;  
pq.push(50);  
pq.push(30);  
pq.push(20);  
pq.push(10);
```

Smallest element is at the top.



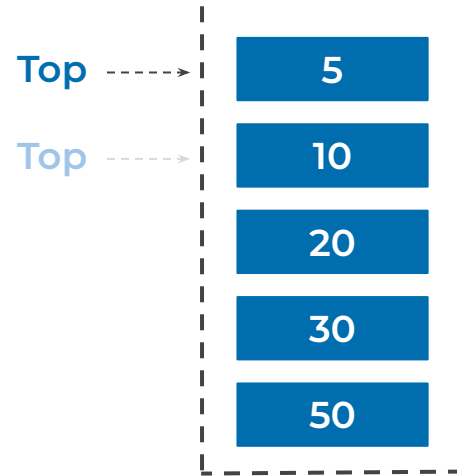
Priority Queue (Min-Heap)

Insert Elements

C++

```
priority_queue<int, vector<int>, greater<int>> pq;  
pq.push(50);  
pq.push(30);  
pq.push(20);  
pq.push(10);  
pq.push(5);
```

Smallest element is at the top.



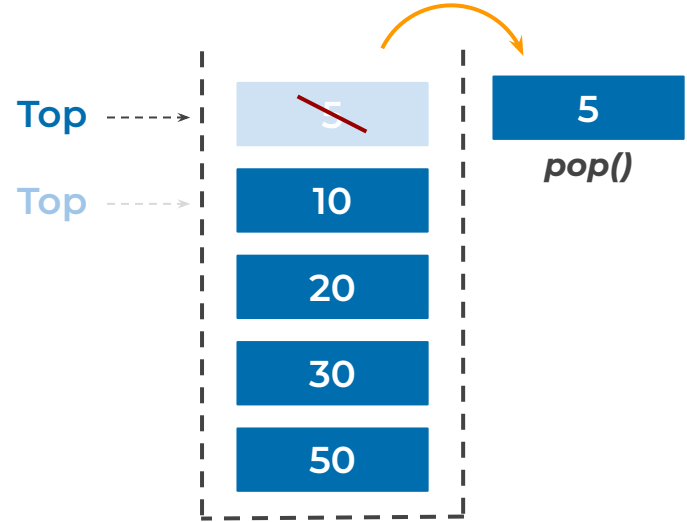
Priority Queue (Min-Heap)

Remove the Top Element

C++ 

```
priority_queue<int, vector<int>, greater<int>> pq;  
.  
.  
.  
pq.pop();
```

Removes the top element "5".



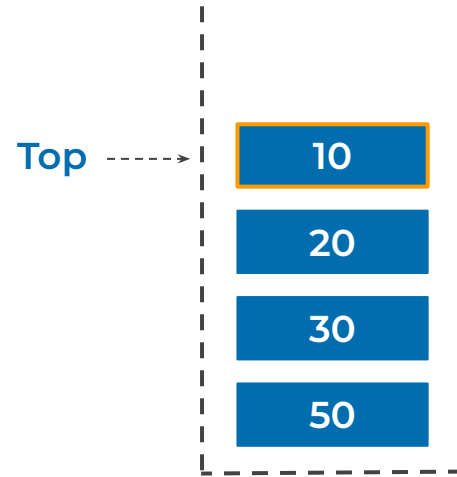
Priority Queue (Min-Heap)

Remove the Top Element

C++ 

```
priority_queue<int, vector<int>, greater<int>> pq;  
.  
.  
.  
cout << pq.top();
```

Returns smallest element "10".



Priority Queue (Min-Heap)

Remove the Top Element

C++ 

```
priority_queue<int, vector<int>, greater<int>> pq;  
.  
.  
.  
cout << pq.top();
```

Returns smallest element "10".

Output 

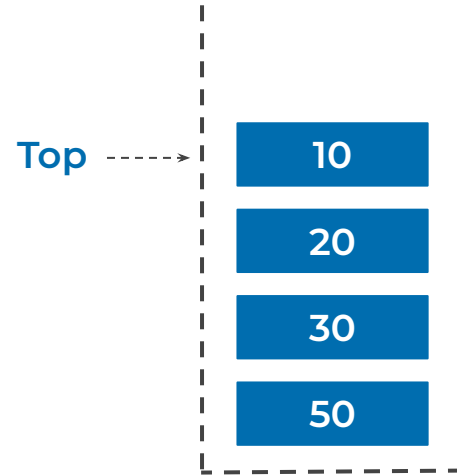
10

Check if the Priority Queue is Empty

C++ 

```
priority_queue<int, vector<int>, greater<int>> pq;  
:  
:  
if (pq.empty()) {  
    cout << "Priority queue is empty";  
}  
else {  
    cout << "Priority queue is not empty";  
}
```

***pq.empty()** returns 1 if it is empty, else returns 0.*



Priority Queue (Min-Heap)

Check if the Priority Queue is Empty

C++ 

```
priority_queue<int, vector<int>, greater<int>> pq;  
:  
:  
if (pq.empty()) {  
    cout << "Priority queue is empty";  
}  
else {  
    cout << "Priority queue is not empty";  
}
```

Output 

Priority queue is not empty

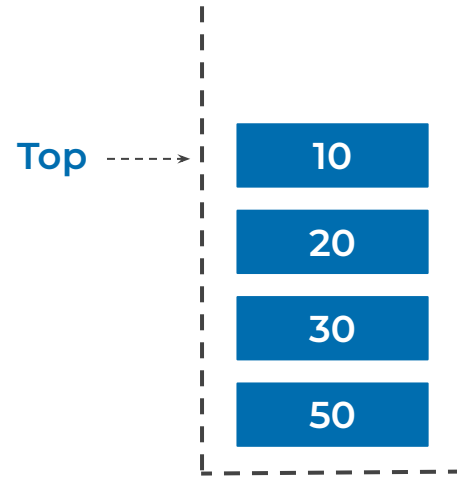
`pq.empty()` returns 1 if it is empty, else returns 0.

Get the Size of the Priority Queue

C++ 

```
priority_queue<int, vector<int>, greater<int>> pq;  
.  
.  
.  
cout << pq.size();
```

Outputs the number of elements in the priority queue.



Priority Queue (Min-Heap)

Get the Size of the Priority Queue

C++ 

```
priority_queue<int, vector<int>, greater<int>> pq;  
.  
.  
.  
cout << pq.size();
```

Output 

4

Outputs the number of elements in the priority queue.

Quiz Time!



Key Takeaways

- ✓ Queue (**FIFO - First In, First Out**): Elements are added at the **rear** and **removed** from the **front**, making it useful for **sequential processing**.
- ✓ **Priority Queue**: A **special queue** where **elements** are **served** based on **priority** rather than **insertion** order. Implemented as **Max-Heap** (**largest** element at the **top**) or **Min-Heap** (**smallest** element at the **top**).



Key Takeaways

✓ *Operations:*

- **Queue:** `push()`, `pop()`, `front()`, `back()`, `size()`, `empty()`.
- **Priority Queue:** `push()`, `pop()`, `top()`, `size()`, `empty()`.

✓ *Examples:*

- **Queue:** Used for task scheduling and **managing requests** in order.
- **Priority Queue:** Used for designing emergency rooms management system (**patients with severe conditions treated first**).





THANK YOU



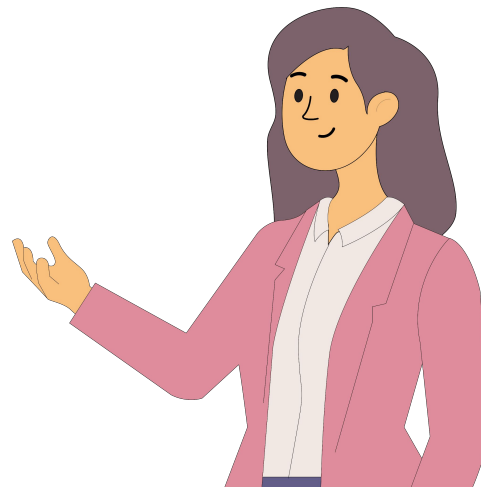


ALL THE BEST



Emergency Room

- ✓ In an emergency room management system, **patients** are **prioritized** based on the **severity** of their condition using a **priority queue**. This ensures that **patients** with the **most critical** conditions **receive immediate attention** and **treatment**, while those with **less severe cases** are handled **subsequently**.
- ✓ Each **patient** is represented by their **name** and **severity level**, where the **severity level** determines their **priority** in the **queue**.



Emergency Room

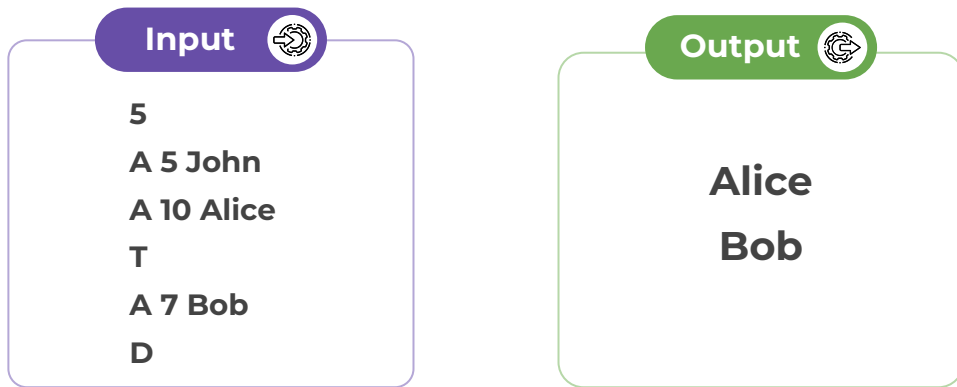
Supported Operations:

1. **Add Patient: A(severity, name)**
 - ✓ Adds a new patient with the specified severity and name to the queue.
2. **Treat and Remove Patient: T()**
 - ✓ Displays and removes the patient with the highest severity from the queue.
3. **Display Next Patient: D()**
 - ✓ Displays the patient with the highest severity without removing them from the queue.



No two patients will have the same severity or name at the same time.

Example



- **A 5 John** : Adds patient **John** with severity **5** to the queue.
- **A 10 Alice** : Adds patient **Alice** with severity **10** to the queue.
- **T** : Removes and treats the patient with the highest severity which is **Alice**.
- **A 7 Bob** : Adds patient **Bob** with severity **7** to the queue.
- **D** : Displays the next patient to be treated, which is **Bob**.

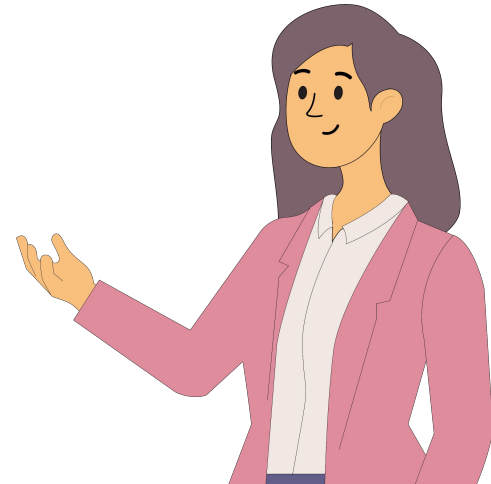
Code

C++ 

```
#include <bits/stdc++.h>
using namespace std;

class EmergencyRoom {
public:
    priority_queue<pair<int, string>> patients;

    void addPatient(string name, int severity) {
        patients.push({severity, name});
    }
}
```



Code

C++



```
void treatPatient() {  
    if (!patients.empty()) {  
        pair<int, string> topPatient = patients.top();  
        patients.pop();  
        cout << patients.top().second << endl;  
    } else {  
        cout << "No patient to treat." << endl;  
    }  
}
```

Code

C++



```
void displayNextPatient() {  
    if (patients.empty()) {  
        cout << "No patient in the queue." << endl;  
        return;  
    }  
    cout << patients.top().second << endl;  
}  
};
```